

摘要：

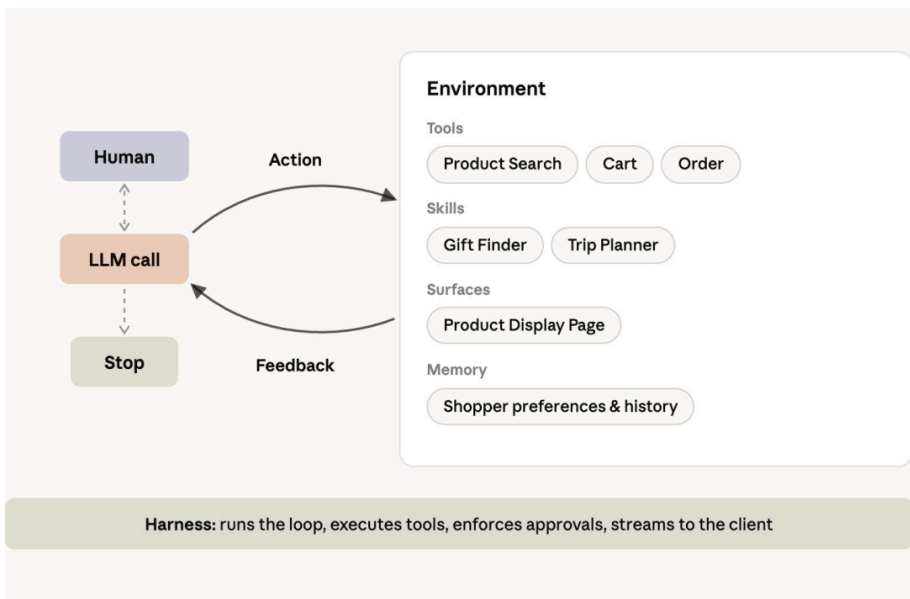
接入这套方案的商家，用户购物车容量直接暴涨了35%，最终完成购买的概率更是狂飙了整整60%。

Anthropic这次直接放大招，把自家的Claude Commerce Agents全套架构和代码开源了！

这是一套直接在零售、旅游、电信和票务现场跑通的硬核落地蓝图，完全甩开了市面上常见的概念演示。实测数据非常炸裂：接入这套方案的商家，用户购物车容量直接暴涨了35%，最终完成购买的概率更是狂飙了整整60%。到底什么样的架构能把AI买卖搞得这么丝滑？Anthropic官方把整套系统的骨架、降本提速技巧以及防止翻车的工程实操全盘托出，干货直接拉满。

核心架构：多智能体拆分是个大坑

面对复杂的电商业务，工程师最容易犯的错误就是按业务拆分出一堆子代理，比如搜商品的、管售后的、算优惠的各来一个，前面再架一个意图路由器。在真实交易场景下，多智能体协同的效果往往极差。电商对话是一个上下文强耦合的过程，购物车数据、用户偏好、历史记录高度交织。每次把任务从主调度器交接给子代理，都会丢失大量上下文状态，不仅严重影响回复质量，还会白白浪费几倍的Token，平白增加几秒钟的响应延迟。各个业务域很难做到彻底切割。一个退换货请求往往需要同时翻看订单历史、当前购物车和商品目录。子代理方案要么让所有模块重复拉取这些数据，要么就得在任务中途来回倒手。

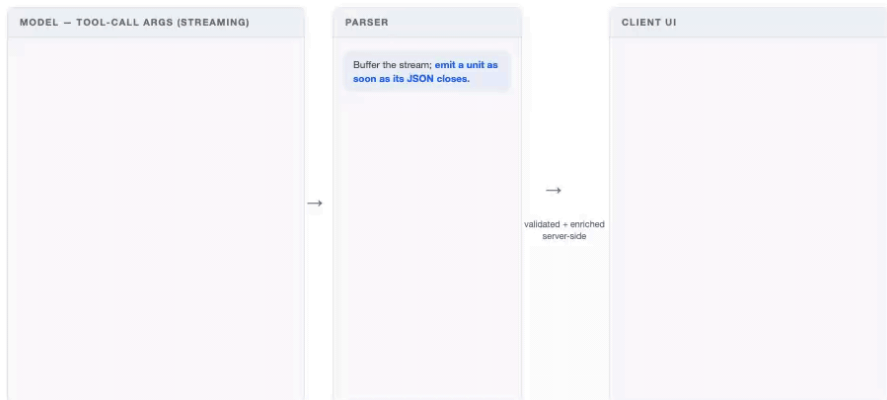


架构图

Anthropic给出的方案极其精简：一个主模型，跑在标准的推理、行动、观察循环里，通过挂载Agent技能来扩展能力。技能说明会直接加载到拥有完整历史记录的主Agent中，完全没有上下文交接损耗。只有两种情况适合引入子代理：

1. 任务高度封闭且重度消耗上下文，比如让专门的深度调研子代理去翻文档、跑代码、查数据，最后只给主模型返回一段精简结论。
2. 业务本身已有独立且合规要求极高的成熟代理系统，此时采用彻底的直接移交，让专用代理接管对话，而不是在一次会话里来回横跳。

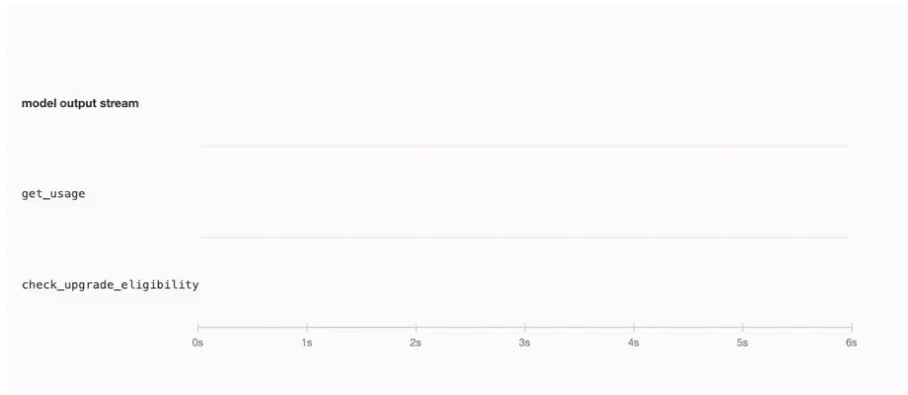
关于指令该写进系统提示词还是做成技能，判断标准只有一条：出现频率。凡是三分之一以上流量都会用到的通用能力，比如基础商品搜索、购物车规则、核心安全合规指令，统统直接写进系统提示词。长尾功能则打包成技能按需加载。如果系统能提前从用户来源页面预判需求，就直接在代码层把技能提前注入，省掉模型自己判断的一轮对话耗时。在工具设计上，UI组件本身就是工具。不要让大模型输出自定义标签再由前端解析，这种做法在复杂场景下极易出错。把商品卡片、行程列表、比价图表封装成具体的展示工具，让模型传入结构化参数，服务器校验后再渲染。这样不仅能保证数据格式百分之百正确，历史记录还能直接作为上下文，让模型准确理解用户说的左边第三个到底对应哪件商品。



展示工具演示

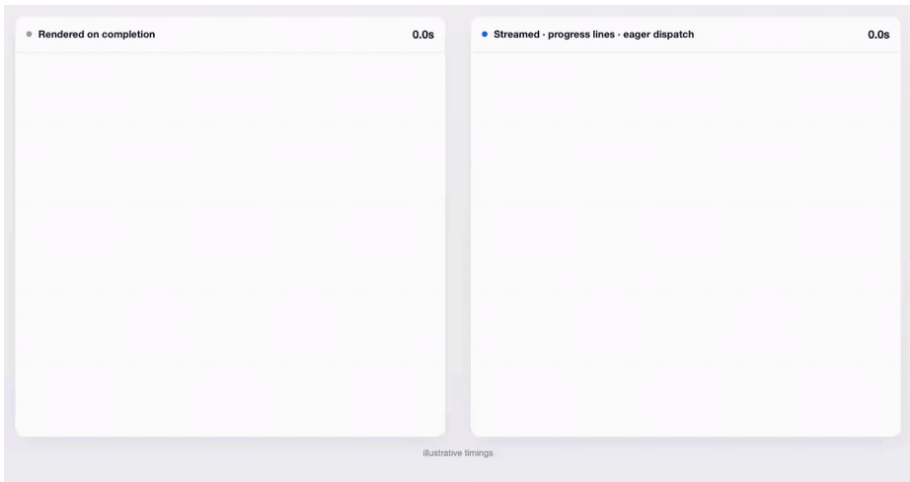
速度与成本：算清真正的任务账本

在电商场景中，用户感知到的最终成效远比单纯压低几毫秒延迟更能决定转化率和留存。完成一个任务的总耗时，等于模型思考轮数、工具处理时间和Token生成时间的总和。想要提速，减少对话轮数往往比单纯换一个生成速度快的弱模型更管用。实测表明，更聪明的模型虽然单字生成稍慢，但规划能力极强，调用工具更精准，可以用更少的交互轮数搞定复杂需求。如果后台数据显示一个任务平均要磨蹭五轮以上，直接换高智商模型往往反而更快。在工程实现上，模型生成工具参数时，代码层可以同步开始执行后端请求，不需要等所有参数完整输出再动身。这就是预先调度机制，能把原本数秒的等待抹平到几百毫秒。



预先调度演示

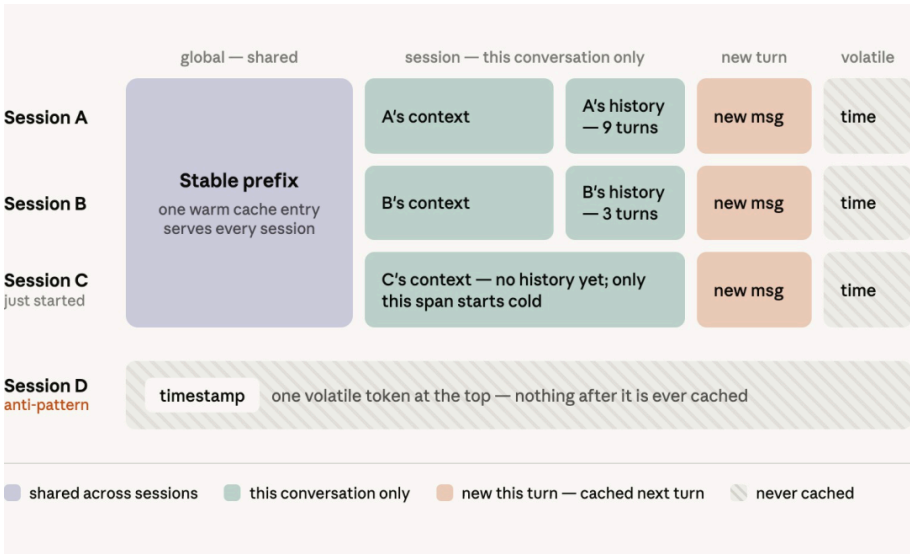
为了降低用户的等待焦虑，前端需要流式渲染组件，并在模型调取背景数据时，实时吐出正在寻找靠海酒店这样的人话进度条，让用户明确看到系统正在干活。



体感延迟对比

省钱的大杀器是提示词缓存。电商场景下，缓存命中率完全可以做到90%到99%。上下文必须严格分层排列：

- 1. 全局层：系统提示词和工具定义放在最前面，全量用户共享，保持字节级一致。
- 2. 会话层：用户画像与历史对话放在中间。
- 3. 易变层：当前时间戳、当前所在网页等每秒都在变的信息，坚决扔到请求的最末尾。把时间戳直接塞在系统提示词开头会导致整个缓存彻底失效。



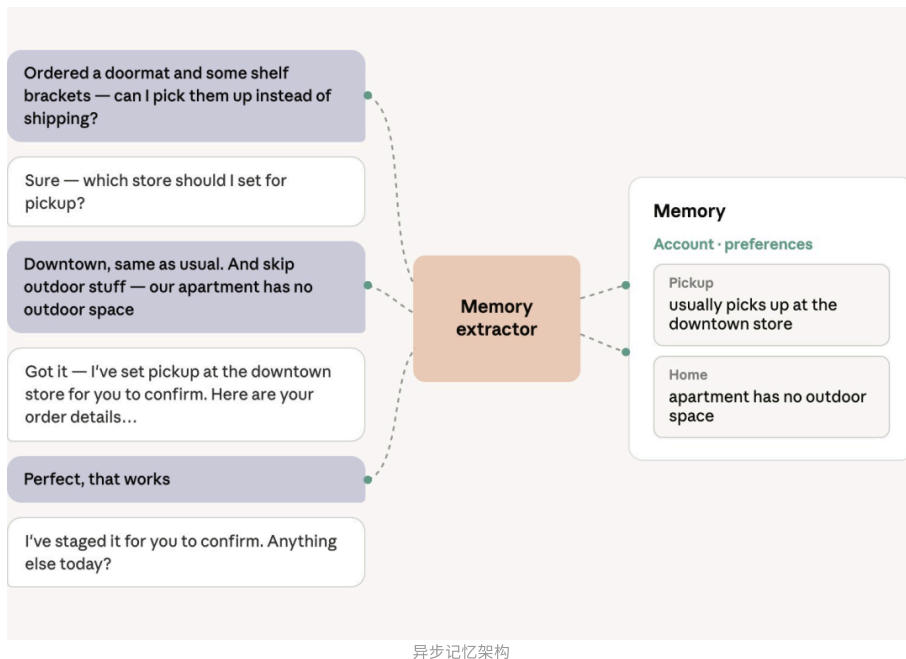
缓存设计示意图

生产落地：防翻车、存记忆与搞评测

让Agent真正跑在生产环境，必须解决三个硬核问题：跨会话记忆、确定性安全防护以及大规模评测。

1. 记忆要异步存进数据库

长期记忆不能依赖模型自己发挥，必须落在传统数据库里，变成一条条带类型的键值记录。针对B端商家运营，记忆必须绑定到具体的登录人而不是共用的大账号，防止权限穿透。写入记忆必须采用异步机制。在用户交互的背后，启动一个独立的后台进程去扫描对话文本，专门提取过敏史、偏好、尺码等事实，默默写入数据库。这种做法完全不占用主对话的响应时间，在实际评测中事实召回率还提升了13%。负责提取的后台模型只阅读纯文本对话，严禁读取商品工具返回的冗余信息，防止把商品介绍误当成用户个人特征。



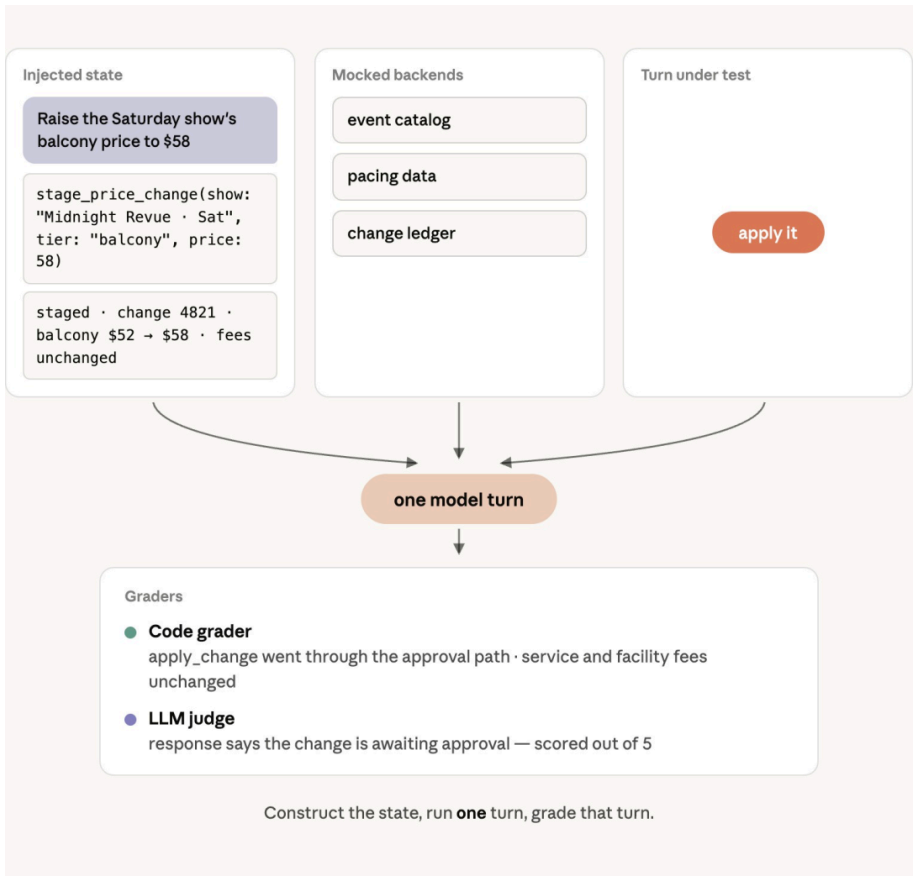
异步记忆架构

2. 安全合规必须写死在代码里

提示词防不住恶意攻击，任何涉及真金白银的操作，底线全部由代码死死锁住。所有修改操作必须分步确认。模型永远只有提议权，没有决定权。下单、扣款、改价、上线营销活动，模型调用的接口只能生成一个待审批编号，最终必须通过用户点击界面按钮或商家在后台确认才能真正执行。模型提交的所有数据ID必须全量校验。系统会记录当前会话里后端真实下发过的所有商品ID和活动ID。只要模型提交了任何一个未曾下发过的凭空捏造ID，后端直接无情拦截。针对限购和降价幅度限制，服务器端强制按最终聚合结果做校验，并对单用户的写入操作进行串行化排队，彻底封死大模型并发调用工具刷单的漏洞。所有来自商品评价、第三方卖家描述的外部文字，在喂给模型前必须经过数据清洗，用固定标签隔开，并在底层明确告诉模型：这些是参考资料，绝对不能当作执行指令。

3. 评测看切片，不要搞双模型过家家

评估Agent时，别用一个AI扮演用户、另一个AI扮演裁判去跑整场模拟对话。两个非确定性系统混在一起，成本高昂且极难定位问题根源。最有效的做法是构建状态切片：把对话历史、当前购物车状态和测试问题一次性拼装好，直接丢给Agent跑一步，然后直接检查它输出的最终状态和工具参数对不对。



评测架构

评测集里不能全是一问一答的标准好题，必须塞满包含长文本、前后矛盾、带有攻击性指令的复杂脏数据。每一个正向测试用例，都必须配一个对应的反向拒绝用例。对于跨业务组合场景，比如商家询问降价15%库存够不够卖，必须同时评测调价提案与库存推演两个动作，单测任何一边都毫无意义。在大团队协作中，谁负责业务系统，谁就认领对应的Agent工具、技能和专属评测集。代码提交走精准CI流水线，改了哪个技能就跑哪个技能及其边界用例。全量评测放在夜间运行，上线前配合灰度切流与一键降级开关，保障核心业务平稳落地。这套架构的核心逻辑很清楚：业务系统还是原来的业务系统，规则也还是原本的业务规则。大模型只负责在最前端做理解、调度与交互。随着基础模型越来越聪明，底层的工程支架只要搭得足够扎实，后续升级模型就只是一次简单的参数调整与评测过筛。

本文来源：[AI寒武纪](#)

风险提示及免责条款

市场有风险，投资需谨慎。本文不构成个人投资建议，也未考虑到个别用户特殊的投资目标、财务状况或需要。用户应考虑本文中的任何意见、观点或结论是否符合其特定状况。据此投资，责任自负。

限时权益申领

* 体验资格每人限申领一次

扫码

免费领取

VIP会员·7天畅读卡 | 大师课·入门串讲



限免

135 收藏

分享到:

写评论

请发表您的评论





表情



图片

发布评论

